

Lab Report

Date: June 12th, 2026

Written by: Yongyu Xu

Team: E1

Team Members

Last name	First name	Email
Rahman	Md Mahbubur	md-mahbubur.rahman@stud.hshl.de
Sakib	Md Naimur Rahaman	md-naimur-rahaman.sakib@stud.hshl.de
Xu	Yongyu	yongyu.xu@stud.hshl.de
Koluacik	Baris	baris.koluacik@stud.hshl.de

Goal

1. The goal of this experiment was to test and improve the performance of a line-following vehicle after code modifications made during the previous week.
2. The experiment focused on identifying hardware and software issues that prevented the vehicle from operating correctly and implementing solutions to achieve stable line-following behavior.

Discussion

1. Motor Controller Wiring Issue

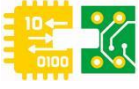
After connecting the power supply and turning on the switch, we discovered that the vehicle could not start moving.

Following a systematic inspection, we found that the motor controller had been connected incorrectly. As a result, the motors were not receiving sufficient voltage to operate properly.

After correcting the wiring, the motors were able to start and the wheels began to rotate. However, the wheel movement was not smooth. Upon further examination, we noticed that some excess material on the motor holders was physically obstructing the wheels. We removed the unnecessary portions of the motor holders, which allowed the wheels to rotate freely and smoothly.

2. Damaged Arduino Board

After resolving the motor issue, we powered on the vehicle again. This time, we noticed that the



Arduino board's power indicator LED did not light up.

Through testing, we confirmed that the Arduino board had been damaged and was no longer functioning. Therefore, we replaced it with a new Arduino and reconnected all the components according to the circuit design.

3. Incorrect Line-Following Logic

Once all hardware components were confirmed to be functioning correctly and all connections had been verified, we placed the vehicle on the black line track and powered it on.

We observed that whenever the vehicle detected the black line, it immediately moved away from the line and then continued driving straight instead of following the track.

After repeated testing and debugging, we determined that the problem originated from an error in the line-following control logic.

Original Logic:

Both IR sensors detect white → Move forward

Left IR sensor detects white, right IR sensor detects black → Turn left

Left IR sensor detects black, right IR sensor detects white → Turn right

This logic caused the vehicle to steer in the wrong direction whenever it detected the line.

Revised Logic:

Both IR sensors detect white → Move forward

Left IR sensor detects white, right IR sensor detects black → Turn right

Left IR sensor detects black, right IR sensor detects white → Turn left

After reversing the turning directions, the vehicle was able to respond correctly to line detection.

4. Additional Software Optimization

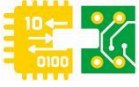
Although the corrected logic allowed the vehicle to turn when it first detected the black line, the vehicle often stopped moving shortly afterward.

A detailed code review revealed several areas that required improvement.

4.1 Replacing delay() with millis()

The original program relied heavily on the delay() function. This caused the vehicle to pause for relatively long periods, reducing responsiveness.

To improve real-time performance, it was suggested to replace delay() with millis(), which allows timing operations without blocking program execution. This makes the vehicle react faster to sensor



inputs.

4.2 Handling Simultaneous Black-Line Detection

The original code contained the following logic:

```
else {  
    stopMotors();  
}
```

When both sensors detected the black line simultaneously, the vehicle stopped completely in the middle of the track.

To prevent unnecessary stopping, the logic was modified so that the vehicle continues moving forward instead:

```
else {  
    forward();  
}
```

This change improved the continuity of movement and reduced the likelihood of the vehicle becoming stuck.

4.3 Reducing Turning Sharpness

The original turning functions used the full turning speed:

```
turnLeft(turnSpeedLeft);  
turnRight(turnSpeedRight);
```

Sharp turns sometimes caused the vehicle to overshoot the line, especially on curved sections of the track.

To improve steering accuracy, the turning speed was reduced:

```
turnLeft(turnSpeedLeft / 2);  
turnRight(turnSpeedRight / 2);
```

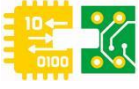
This resulted in smoother and more controlled turns.

4.4 Improving Obstacle Handling

The obstacle detection code originally used:

```
if (rightDistance < obstacleLimit || leftDistance < obstacleLimit) {  
    stopMotors();  
    return;  
}
```

When an obstacle was detected, the vehicle simply stopped and remained stationary.



To improve autonomy, it was suggested that the vehicle perform an avoidance maneuver after stopping rather than remaining inactive. This would allow the vehicle to continue operating without manual intervention.

5. IR Sensor Sensitivity Calibration

After implementing the software improvements, another issue was observed.

The right IR sensor continuously reported detecting a black surface whenever it was placed near the ground. However, when the sensor was lifted above the table, it could correctly distinguish between black and white surfaces.

We hypothesized that the two IR sensors had different sensitivity levels. Therefore, we repeatedly adjusted the sensitivity potentiometers on both sensors and conducted multiple rounds of testing.

After careful calibration, both sensors produced consistent and reliable readings. As a result, the vehicle was finally able to follow the black line smoothly and accurately.

Conclusion

During this experiment, several hardware and software issues were identified and resolved. These included incorrect motor controller wiring, mechanical interference from the motor holders, a damaged Arduino board, incorrect line-following logic, inefficient program structure, and mismatched IR sensor sensitivity.

Through systematic troubleshooting, code optimization, and sensor calibration, the vehicle's performance was significantly improved. The vehicle was able to detect the line accurately and follow the track smoothly and reliably.